3C

DESARROLLO DE SOFTWARE

Desarrollo de aplicaciones de Software para la solución de problemas

Saber, valorar, elegir

PADRE ARRUPE
Colegio Español

DOCENTE: Daniel Sosa

COLEGIO ESPAÑOL PADRE ARRUPE

Colegio internacional.

Tercer Año de Bachillerato de Software

Desarrollo de aplicaciones de Software para la solución de problemas

Periodo 1.

GUÍA 2 – Creación y consumo de microservicios

OBJETIVOS DE LA PRÁCTICA

Objetivo general

- Comprender y aplicar los fundamentos de las arquitecturas basadas en microservicios mediante la construcción y consumo de una API REST en .NET Core, conectada a una base de datos SQL Server y consumida desde un cliente PHP.

Objetivos específicos

1. Identificar las diferencias entre aplicaciones monolíticas y arquitecturas de microservicios.
2. Configurar una base de datos en SQL Server y crear la tabla Products.
3. Implementar un servicio REST en ASP.NET Core que exponga operaciones CRUD sobre la tabla.
4. Probar los endpoints utilizando Postman para validar su funcionamiento.
5. Consumir la API desde un cliente PHP, mostrando los datos en una interfaz sencilla.
6. Analizar las ventajas y desventajas de los microservicios en comparación con arquitecturas tradicionales.
7. Documentar el proceso paso a paso para reforzar la comprensión y facilitar la replicación del ejercicio.

INTRODUCCIÓN TEÓRICA

Los **microservicios** son un estilo de arquitectura de software que divide una aplicación en servicios pequeños, autónomos y especializados. Cada microservicio se centra en una única responsabilidad y se comunica con otros mediante protocolos ligeros como **HTTP/REST** o sistemas de mensajería.

En contraste con las aplicaciones **monolíticas**, donde toda la lógica está empaquetada en un solo bloque, los microservicios permiten:

- **Escalabilidad independiente**: cada servicio puede crecer según la demanda.
- **Flexibilidad tecnológica**: distintos servicios pueden implementarse en diferentes lenguajes o frameworks.
- **Resiliencia**: si un servicio falla, los demás pueden seguir funcionando.
- **Agilidad en el desarrollo**: equipos pequeños pueden trabajar en paralelo en diferentes servicios.

Sin embargo, también presentan retos: mayor complejidad arquitectónica, necesidad de infraestructura avanzada (contenedores, orquestadores como Kubernetes), y pruebas de integración más exigentes.

El ejercicio **DemoApi1** ejemplifica este enfoque:

- **SQL Server** como almacenamiento de datos.
- **ASP.NET Core** para exponer la lógica de negocio mediante una API REST.
- **PHP** como cliente externo que consume los datos.

REALIZACIÓN DE PRÁCTICA

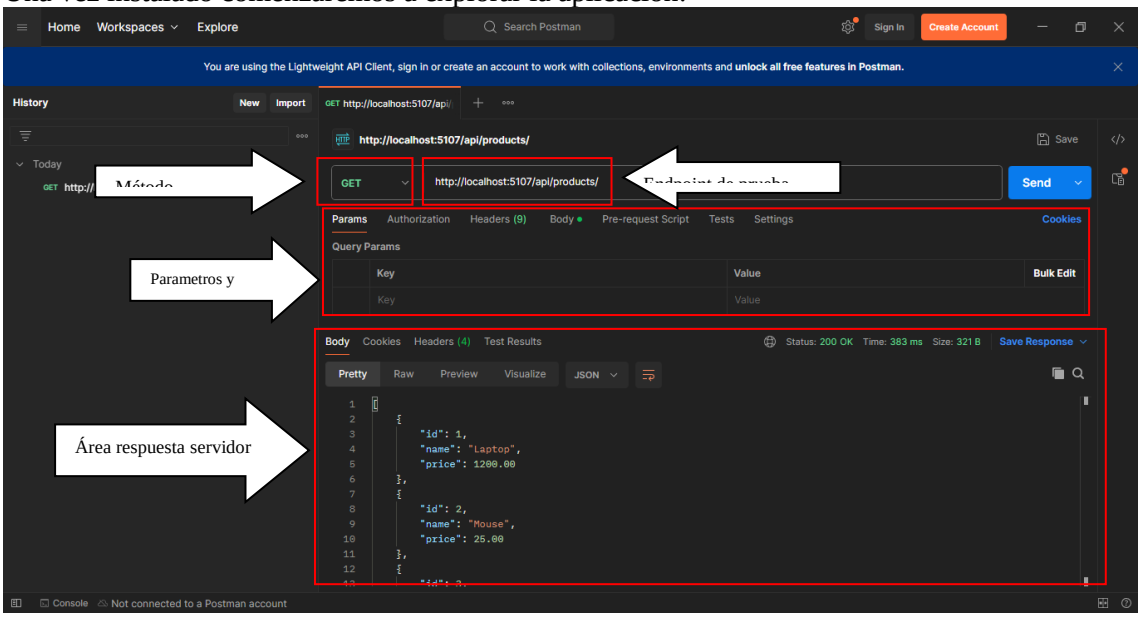
Para la primera parte de la guía vamos a agregar los siguientes métodos en el controlador a nuestro proyecto DemoApi

```
        // PUT: api/products/1
        [HttpPut("{id}")]
        0 referencias
        public IActionResult UpdateProduct(int id, Product product)
        {
            if (id != product.Id) return BadRequest();

            var existingProduct = _context.Products.Find(id);
            if (existingProduct == null) return NotFound();

            existingProduct.Name = product.Name;
            existingProduct.Price = product.Price;

            _context.SaveChanges();
            return Ok(existingProduct);
        }

        // PATCH: api/products/1
        [HttpPatch("{id}")]
        0 referencias
        public IActionResult UpdateProductPartial(int id, [FromBody] Product product)
        {
            var existingProduct = _context.Products.Find(id);
            if (existingProduct == null) return NotFound();

            if (!string.IsNullOrEmpty(product.Name))
                existingProduct.Name = product.Name;

            if (product.Price > 0)
                existingProduct.Price = product.Price;

            _context.SaveChanges();
            return Ok(existingProduct);
        }

        // DELETE: api/products/1
        [HttpDelete("{id}")]
        0 referencias
        public IActionResult DeleteProduct(int id)
        {
            var product = _context.Products.Find(id);
            if (product == null) return NotFound();

            _context.Products.Remove(product);
            _context.SaveChanges();
            return NoContent();
        }
    }
}
```

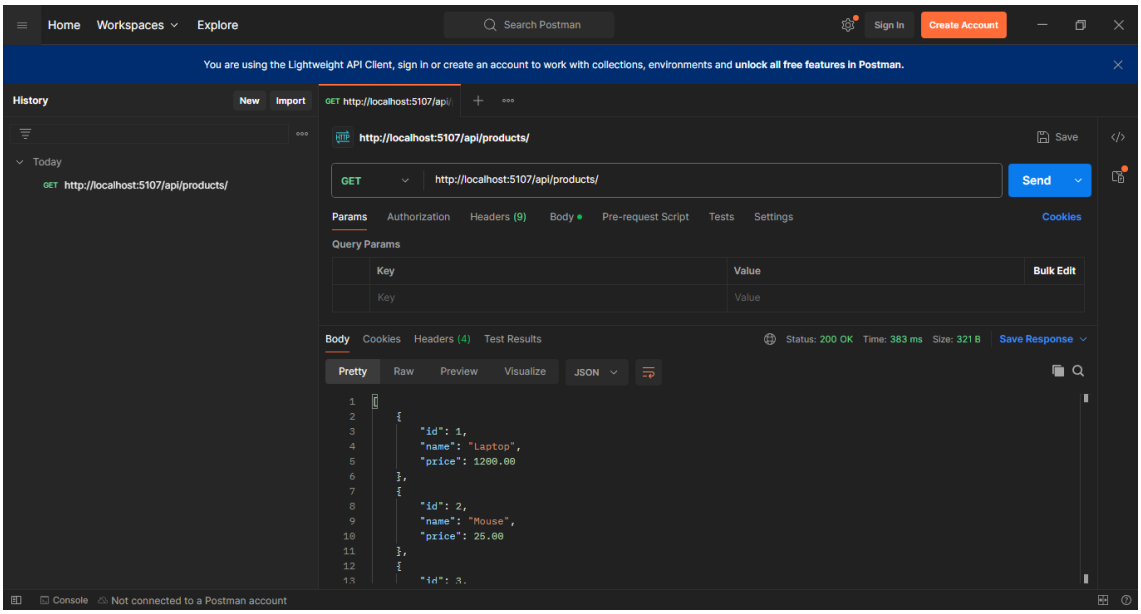
1. Vamos a descargar Postman como herramienta de pruebas para nuestro endpoint del proyecto DemoApi

https://www.postman.com/downloads/

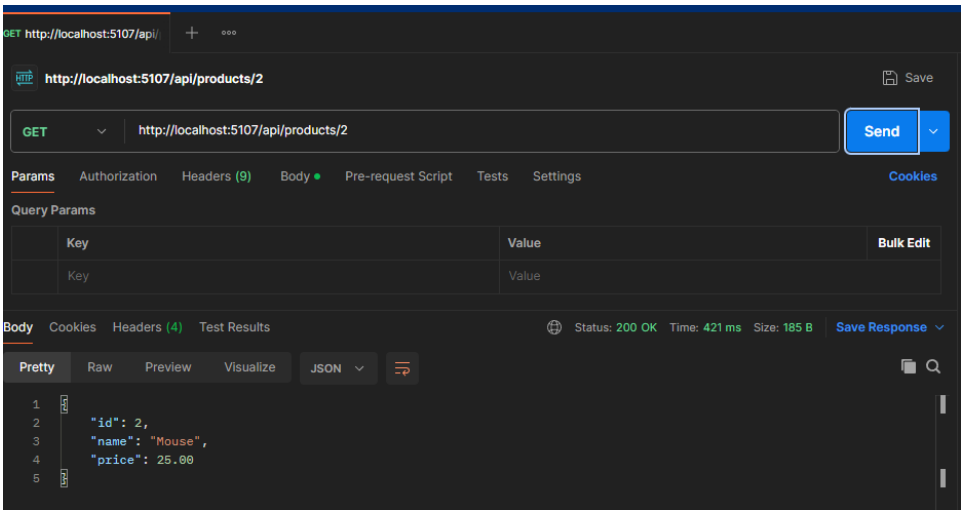
Una vez instalado comenzaremos a explorar la aplicación:



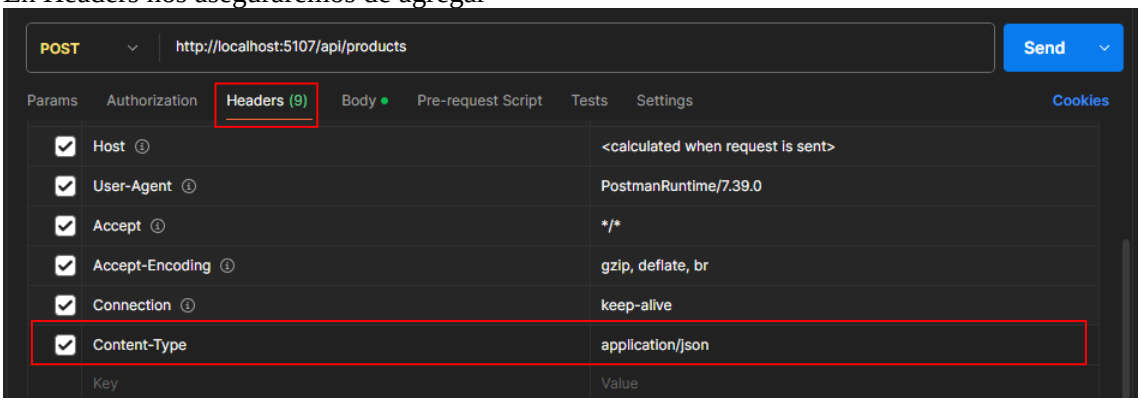
Método GET: Observamos que al realizar la petición nos devuelve de forma correcta el json



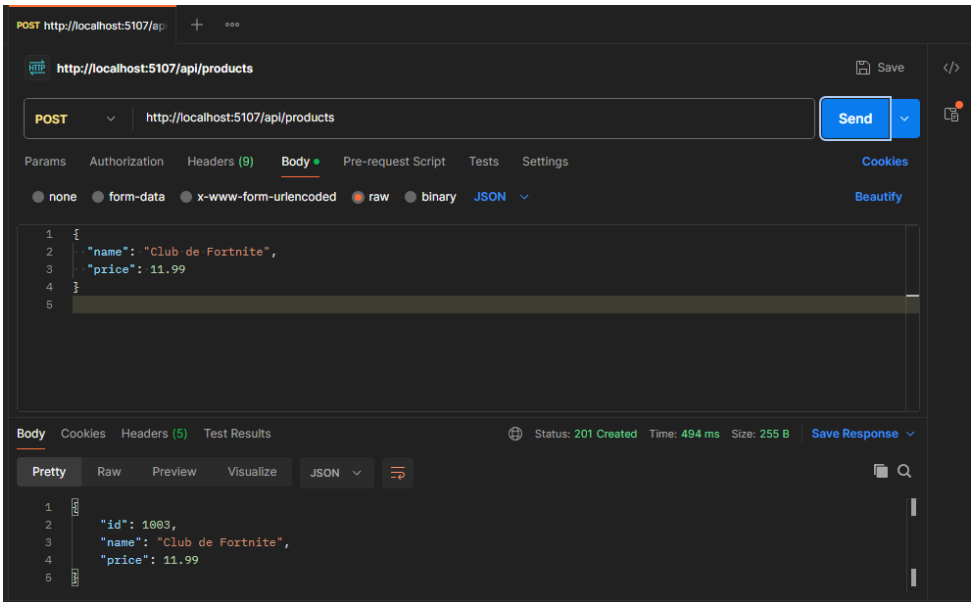
por Id:



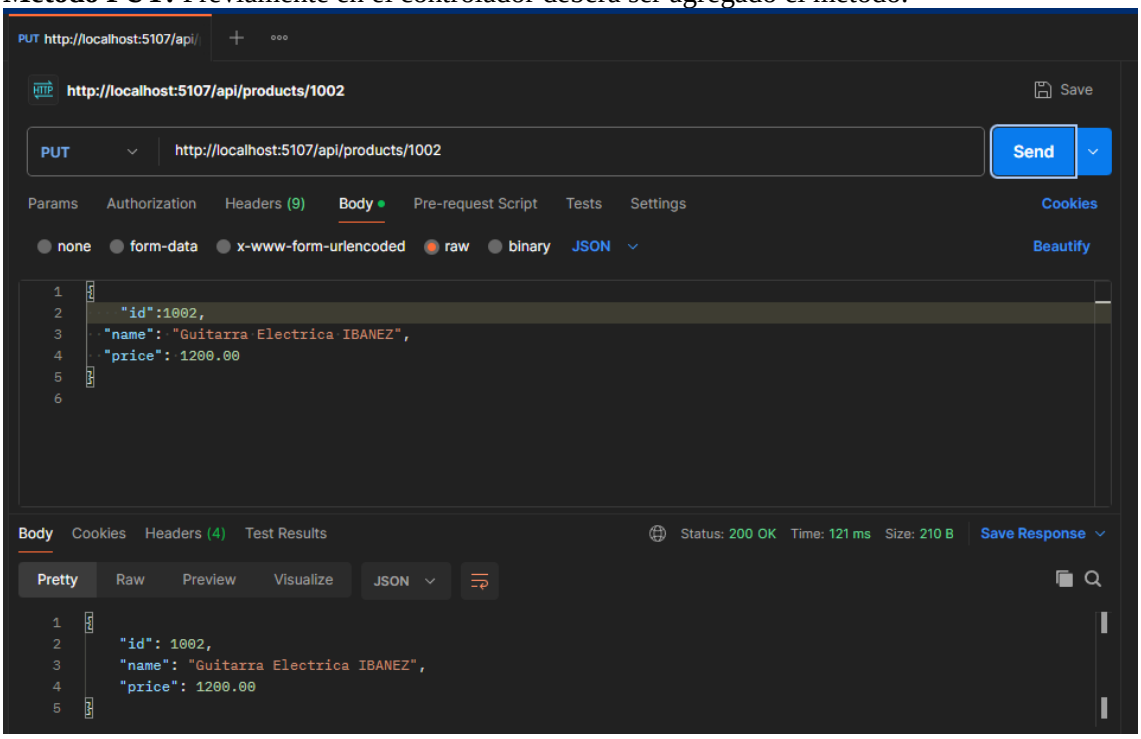
Método POST: Observamos que al realizar la petición nos devuelve de forma correcta el json
Para el método POST vamos a agregar la siguiente configuración:
En Headers nos aseguraremos de agregar



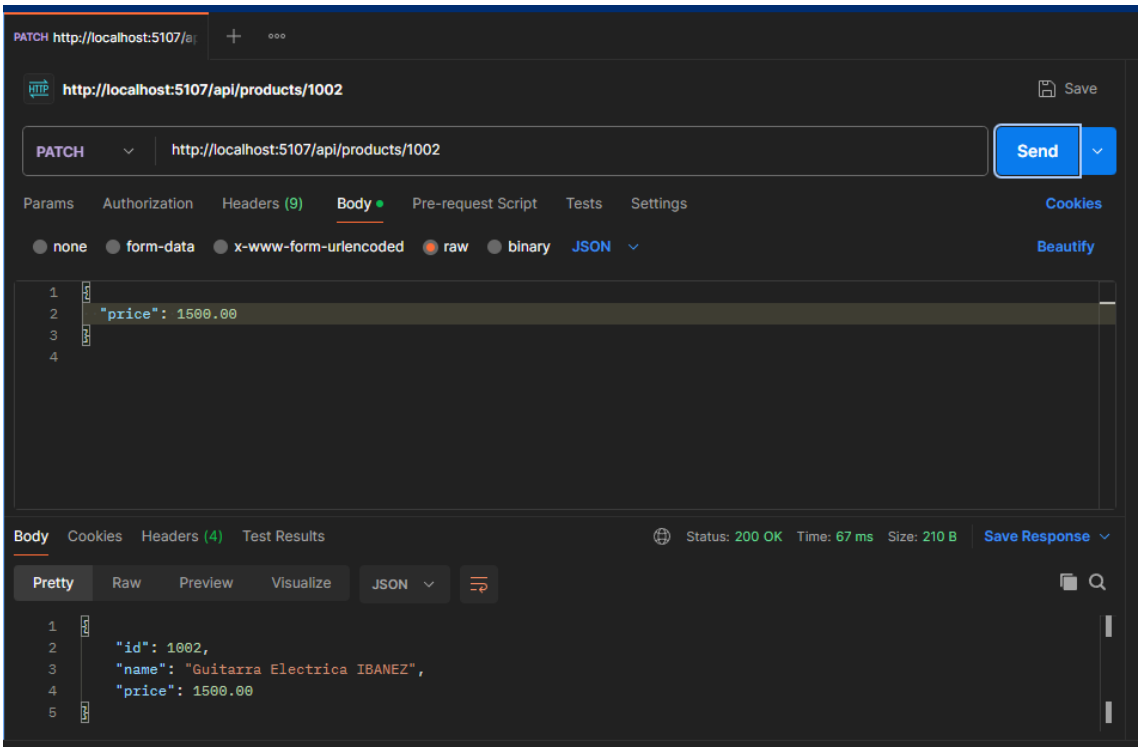
Luego nos vamos a Body>RAW y creamos el json con los valores a insertar según los campos en la bdd, si todo está escrito y configurado de forma correcta el estado de la petición será 201



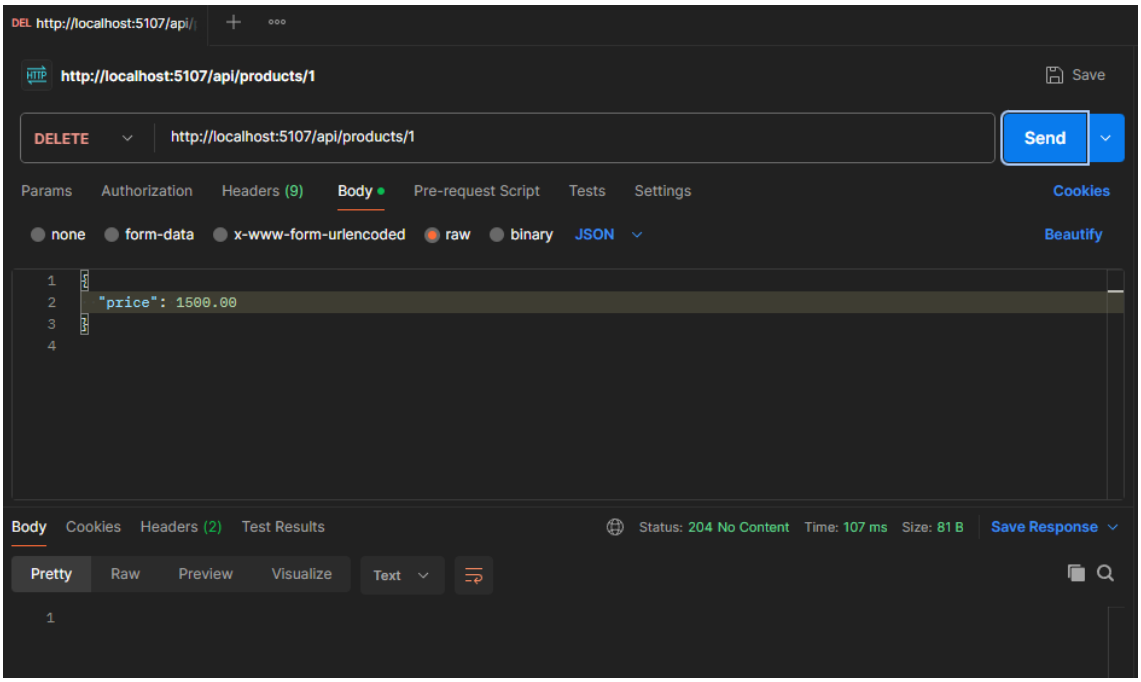
Método PUT: Previamente en el controlador deberá ser agregado el método.



Método PATCH:



Método DELETE: Para probar si funciono después hacer un GET y verificar que ya no esté el producto con el id eliminado



Con esta parte hemos podido comprobar el correcto funcionamiento de nuestro CRUD sin necesidad de hacer mayor implementación a nivel de proyecto.

Segunda parte de la Guía, creación de vistas a través de .NET Core MVC

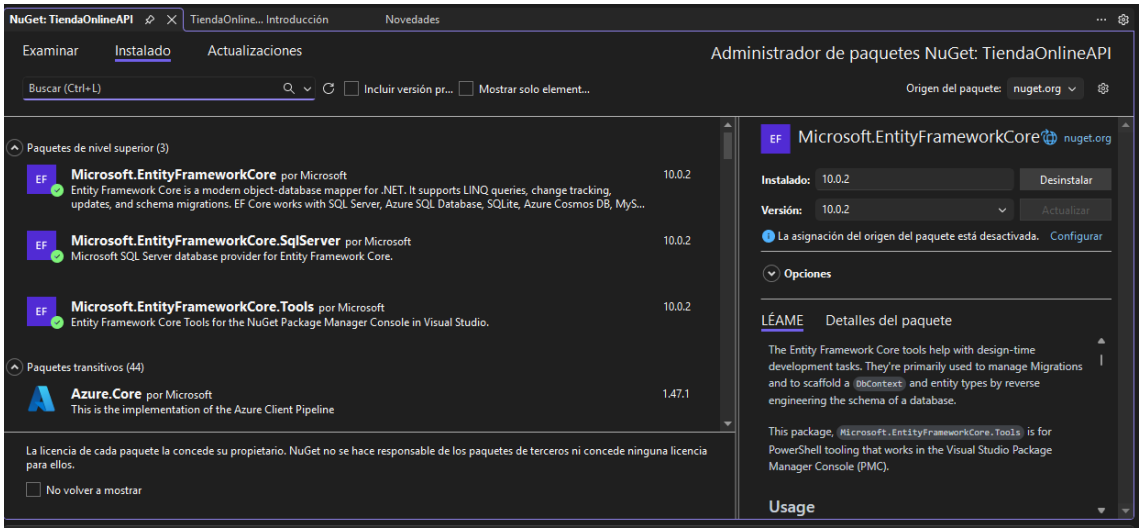
Creando la base de datos ‘TiendaOnline’

```sql
-- Crear base de datos
CREATE DATABASE TiendaOnline;
GO

USE TiendaOnline;
GO

-- Tabla Categorias
CREATE TABLE Categorias (
    CategoriaId INT PRIMARY KEY IDENTITY(1,1),
    Nombre NVARCHAR(100) NOT NULL
);

-- Tabla Productos
CREATE TABLE Productos (
    ProductoId INT PRIMARY KEY IDENTITY(1,1),
    Nombre NVARCHAR(200) NOT NULL,
    Descripcion NVARCHAR(500),
    Precio DECIMAL(10,2) NOT NULL,
    UrlImagen NVARCHAR(500),
    CategoriaId INT FOREIGN KEY REFERENCES Categorias(CategoriaId)
);

-- Tabla Pedidos
CREATE TABLE Pedidos (
    PedidoId INT PRIMARY KEY IDENTITY(1,1),
    FechaPedido DATETIME DEFAULT GETDATE(),
    Cliente NVARCHAR(200) NOT NULL
);

-- Tabla Detalle de Pedido
CREATE TABLE PedidoDetalle (
    DetalleId INT PRIMARY KEY IDENTITY(1,1),
    PedidoId INT FOREIGN KEY REFERENCES Pedidos(PedidoId),
    ProductoId INT FOREIGN KEY REFERENCES Productos(ProductoId),
    Cantidad INT NOT NULL,
    PrecioUnitario DECIMAL(10,2) NOT NULL
);
```

Insertando datos de prueba:

```sql
-- Insertando datos
INSERT INTO Categorias (Nombre) VALUES ('Electrónica'), ('Ropa'), ('Hogar');

INSERT INTO Productos (Nombre, Descripcion, Precio, UrlImagen, CategoriaId)
VALUES
('Laptop Dell', 'Laptop de 15 pulgadas', 850.00, 'https://example.com/laptop.jpg', 1),
('Camiseta Negra', 'Camiseta de algodón', 15.00, 'https://example.com/camiseta.jpg', 2),
('Silla Gamer', 'Silla ergonómica para juegos', 120.00, 'https://example.com/silla.jpg', 3);
```

Configuración proyecto:
Nombre: TiendaOnlineAPI
Framework: 10.0
NuGets instalados:



Ahora vamos a crear las siguientes clases:
Configurar el appsettings.json para la correcta conexión a la base de datos
Models/TiendaContext.cs

```
using Microsoft.EntityFrameworkCore;
using System.ComponentModel.DataAnnotations;

namespace TiendaOnlineAPI.Models
{
    5 referencias
    public class TiendaContext : DbContext
    {
        0 referencias
        public TiendaContext(DbContextOptions<TiendaContext> options) : base(options) { }

        0 referencias
        public DbSet<Categoria> Categorias { get; set; }
        3 referencias
        public DbSet<Producto> Productos { get; set; }
        0 referencias
        public DbSet<Pedido> Pedidos { get; set; }
        0 referencias
        public DbSet<PedidoDetalle> PedidoDetalles { get; set; }

        0 referencias
        protected override void OnModelCreating(ModelBuilder modelBuilder)
        {
            // 👉 Definir clave primaria explícita para PedidoDetalle
            modelBuilder.Entity<PedidoDetalle>()
                .HasKey(pd => pd.PedidoDetalleId);

            base.OnModelCreating(modelBuilder);
        }
    }

    2 referencias
    public class Categoria
    {
        0 referencias
        public int CategoriaId { get; set; }
        0 referencias
        public string Nombre { get; set; }
    }

    6 referencias
    public class Producto
    {
        2 referencias
        public int ProductoId { get; set; }
        0 referencias
        public string Nombre { get; set; }
        0 referencias
        public string Descripcion { get; set; }
        0 referencias
        public decimal Precio { get; set; }
        0 referencias
        public string UrlImagen { get; set; }

        0 referencias
        public int CategoriaId { get; set; }
        2 referencias
        public Categoria Categoria { get; set; }
    }

    2 referencias
    public class Pedido
    {
        0 referencias
        public int PedidoId { get; set; }
        0 referencias
        public DateTime FechaPedido { get; set; }
        0 referencias
        public string Cliente { get; set; }

        0 referencias
        public ICollection<PedidoDetalle> Detalles { get; set; }
    }

    3 referencias
    public class PedidoDetalle
    {
        [Key] // 👉 EF reconocerá esta propiedad como PK
        1 referencia
        public int PedidoDetalleId { get; set; }

        0 referencias
        public int PedidoId { get; set; }
        0 referencias
        public Pedido Pedido { get; set; }

        0 referencias
        public int ProductoId { get; set; }
        0 referencias
        public Producto Producto { get; set; }

        0 referencias
        public int Cantidad { get; set; }
        0 referencias
        public decimal PrecioUnitario { get; set; }
    }
}
```

Controllers/ProductosController.cs

```
using Microsoft.AspNetCore.Mvc;
using Microsoft.EntityFrameworkCore;
using TiendaOnlineAPI.Models;

namespace TiendaOnlineAPI.Controllers
{
    [ApiController]
    [Route("api/[controller]")]
    1 referencia
    public class ProductosController : ControllerBase
    {
        private readonly TiendaContext _context;

        0 referencias
        public ProductosController(TiendaContext context)
        {
            _context = context;
        }

        //GET: api/productos
        [HttpGet]
        0 referencias
        public async Task<ActionResult<IEnumerable<Producto>>> GetProductos()
        {
            return await _context.Productos.Include(p => p.Categoria).ToListAsync();
        }

        //GET: api/productos/{id}
        [HttpGet("{id}")]
        1 referencia
        public async Task<ActionResult<Producto>> GetProducto(int id)
        {
            var producto = await _context.Productos
                .Include(p => p.Categoria)
                .FirstOrDefaultAsync(p => p.ProductoId == id);

            if (producto == null) return NotFound();
            return producto;
        }

    }
}
```

Program.cs

```
using Microsoft.EntityFrameworkCore;
using TiendaOnlineAPI.Models;

var builder = WebApplication.CreateBuilder(args);

// Registrar servicios
builder.Services.AddControllers();
builder.Services.AddDbContext<TiendaContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("TiendaConnection")));

var app = builder.Build();

// Configuración del pipeline
if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseRouting();
app.UseAuthorization();

// Mapear controladores de API
app.MapControllers();

app.Run();
```

Al desplegar la app, el index nos dará un error, pues en realidad no hay una vista dentro de este proyecto, sin embargo si accedemos a los endpoints si tendremos acceso a ellos:

localhost:5235/api/productos

Impresión con sangría


```
[
  {
    "productoId": 1,
    "nombre": "Laptop Dell",
    "descripcion": "Laptop de 15 pulgadas",
    "precio": 850,
    "urlImagen": "https://example.com/laptop.jpg",
    "categoriaId": 1,
    "categoria": {
      "categoriaId": 1,
      "nombre": "Electrónica"
    }
  },
  {
    "productoId": 2,
    "nombre": "Camiseta Negra",
    "descripcion": "Camiseta de algodón",
    "precio": 15,
    "urlImagen": "https://example.com/camiseta.jpg",
    "categoriaId": 2,
    "categoria": {
      "categoriaId": 2,
      "nombre": "Ropa"
    }
  },
  {
    "productoId": 3,
    "nombre": "Silla Gamer",
    "descripcion": "Silla ergonómica para juegos",
    "precio": 120,
    "urlImagen": "https://example.com/silla.jpg",
    "categoriaId": 3,
    "categoria": {
      "categoriaId": 3,
      "nombre": "Hogar"
    }
  }
]
```


Parte 2, creando una vista dentro del proyecto consumiendo la API

En la carpeta Views del proyecto (En caso de no haberse creado deberá crearlas por su cuenta) vamos a agregar la siguiente carpeta **ProductosMvc**

Dentro de esta carpeta deberá crear las vistas **Index.cshtml** e **Create.cshtml**

Index

```
@model IEnumerable<TiendaOnlineAPI.Models.Producto>

<h2>Listado de Productos</h2>

<p>
    <a asp-action="Create" class="btn btn-primary">Agregar Producto</a>
</p>

<table class="table table-striped">
    <thead>
        <tr>
            <th>Nombre</th>
            <th>Descripción</th>
            <th>Precio</th>
            <th>Categoría</th>
            <th>Imagen</th>
        </tr>
    </thead>
    <tbody>
        @foreach (var item in Model)
        {
            <tr>
                <td>@item.Nombre</td>
                <td>@item.Descripcion</td>
                <td>@item.Precio</td>
                <td>@item.Categoria?.Nombre</td>
                <td>
                    @if (!string.IsNullOrEmpty(item.UrlImagen))
                    {
                        <img src="@item.UrlImagen" alt="@item.Nombre" width="100" />
                    }
                </td>
            </tr>
        }
    </tbody>
</table>
```

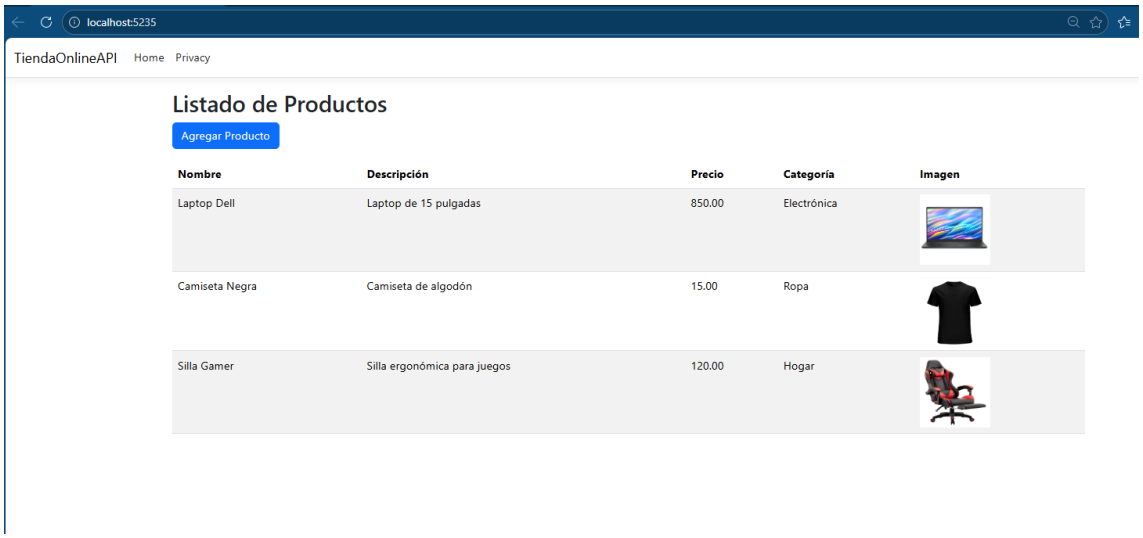
Create

```razor
@model TiendaOnlineAPI.Models.Producto

<h2>Agregar Producto</h2>

<form asp-action="Create" method="post">
    <div class="form-group">
        <label for="Nombre">Nombre</label>
        <input asp-for="Nombre" class="form-control" />
    </div>

    <div class="form-group">
        <label for="Descripcion">Descripción</label>
        <input asp-for="Descripcion" class="form-control" />
    </div>

    <div class="form-group">
        <label for="Precio">Precio</label>
        <input asp-for="Precio" class="form-control" type="number" step="0.01" />
    </div>

    <div class="form-group">
        <label for="UrlImagen">URL Imagen</label>
        <input asp-for="UrlImagen" class="form-control" />
    </div>

    <div class="form-group">
        <label for="CategoriaId">Categoría</label>
        <input asp-for="CategoriaId" class="form-control" type="number" />
    </div>

    <button type="submit" class="btn btn-success">Guardar</button>
</form>
```

Modificaremos Program.cs

```razor
using Microsoft.EntityFrameworkCore;
using TiendaOnlineAPI.Models;

var builder = WebApplication.CreateBuilder(args);

//Registrar servicios MVC con vistas
builder.Services.AddControllersWithViews();

builder.Services.AddDbContext<TiendaContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("TiendaConnection")));

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    app.UseExceptionHandler("/Home/Error");
    app.UseHsts();
}

app.UseHttpsRedirection();
app.UseStaticFiles();   //necesario para CSS, JS, imágenes

app.UseRouting();
app.UseAuthorization();

// Ruta por defecto hacia tu controlador MVC
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=ProductosMvc}/{action=Index}/{id?}");

app.Run();
```

Al desplegar el proyecto debería verse así:



ACTIVIDAD COMPLEMENTARIA

1. Implementar el funcionamiento completo del CRUD:
    a. Deberá modificar el código pertinente
    b. Realizar pruebas en postman evidenciándolas en un documento pdf
2. Implementar las vistas en el proyecto y formularios completos para los métodos: Crear, modificar y eliminar.
    a. Evidenciar en capturas con el pdf del punto anterior

BIBLIOGRAFÍA

- Newman, S. (2021). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns: With examples in Java*. Manning Publications.
- Microsoft Docs: Microservices architecture (learn.microsoft.com in Bing)
- Fowler, M. (2014). *Microservices* (artículo en línea). Disponible en: martinfowler.com (martinfowler.com in Bing)
- Dragoni, N., et al. (2017). *Microservices: Yesterday, Today, and Tomorrow*. En *Present and Ulterior Software Engineering*. Springer.